

Exercise 5: The API Surface: Records, Drafts and the Review Workflow

DEV CYCLE	Develop (API)
CC FEATURES	Build loop, running the live server, validating in the Swagger UI (/docs), pydantic schemas
SPEC TRACES	FR-01...FR-04, FR-07...FR-12, FR-18, FR-19, SEC-01; API table in SPEC §11
TIME	55 min
YOU PRODUCE	A running FastAPI app: submit records, create a (stub) draft, approve / reject under the four-eyes rule, release to patient

Objective

Stand up the API and the clinician-in-the-loop workflow. You will generate pydantic schemas and routes, run the server, and drive the whole journey by hand in Swagger. The AI is deliberately not wired in yet: drafts come from a clearly-marked stub, so you can prove the review workflow is correct on its own before a model adds uncertainty. This is the most satisfying exercise: by the end you have a clickable, rule-enforcing API.

Walk-through

1 Generate request and response schemas

pydantic models are the API contract. Keep them separate from the ORM models so the wire format and the storage format can differ; here they must, because the list view is not allowed to contain PHI.

TYPE THIS PROMPT INTO CLAUDE CODE

Create app/schemas.py with pydantic v2 models per SPEC §5 and §6: MedicationIn/Out; DischargeCreate (all record fields plus a non-empty medications list); DischargeOut (full record); DischargeListItem (id, patient_ref, patient_initials, primary_diagnosis, discharge_date, status, submitted_by: no name, MRN, DOB or phone, FR-03); DraftContent (SPEC §7.2); DraftOut; DraftRequest {requested_by}; ApproveIn {reviewer_id}; RejectIn {reviewer_id, reason with min_length 10}; PatientSummaryOut. Validation per FR-02: dose_x1000 must be a strict positive int (use StrictInt so 2.5 and "500" are rejected), enums for dose_unit, route and change_type, patient_ref matches ^PT-\d{4,}\$, and a model validator that discharge_date >= admission_date.

VALIDATE Check the diff for StrictInt (or an equivalent strict validator) on dose_x1000; plain int would silently coerce "500000". Check that DischargeListItem genuinely has no PHI fields. Check the date validator.

2 Generate the routes, auth and a stub drafter

Now the endpoints and app entry point. Tell Claude exactly what the stub drafter may do, so it cannot quietly fake AI output. The stub fills DraftContent from the record fields with a visible marker; real drafting arrives in Exercise 6.

TYPE THIS PROMPT INTO CLAUDE CODE

Create app/routes.py (APIRouter) and app/main.py (FastAPI app, includes the router, calls init_db on startup via lifespan). Implement every endpoint in SPEC §11 except POST/GET /references and GET /review-queue (later exercises).

Rules: an APIKeyHeader dependency checks X-API-Key against DISCHARGE_API_KEY on every write endpoint (401 otherwise, SEC-01). Draft creation (FR-04) computes the next version, sets the draft to pending_review and the record to awaiting_review, and returns 409 if a pending draft exists or the record is approved. For now build DraftContent with a function stub_draft(record) that copies the record's own text into the fields and prefixes greeting with "[STUB] "; mark it # TODO(Ex6): replace with Claude drafting. Render medication_table from the Medication rows, formatting dose_x1000 back to a human dose (500000 mg -> "500 mg") for display only (FR-05).

Approve and reject enforce FR-08 to FR-11: 403 if reviewer_id == record.submitted_by, 409 unless the draft is pending_review, reject requires a reason. patient-summary returns 409 unless the record is approved and never includes MRN, DOB or phone (FR-12). Write an audit_events row for each state change with no PHI (FR-18). Add GET /health (FR-19). Every endpoint docstring cites its FR ID.

VALIDATE Read the diff across main.py and routes.py. Look for: the TODO(Ex6) stub; the four-eyes comparison; the status guard before any update; audit rows that use patient_ref, never patient_name; and dose formatting that divides by 1000 only at render time. Approve the changes.

SCREENSHOT SLOT Diff showing the four-eyes check and the 409 status guard

3 Run the live server

Start Uvicorn with auto-reload. Leave this terminal running and open a second terminal (with the venv activated) for everything else.

RUN IN TERMINAL

```
PS> python -m uvicorn app.main:app --reload
```

VALIDATE The terminal shows Uvicorn running on http://127.0.0.1:8000 and Application startup complete. Any traceback here is your next prompt to Claude: paste it whole.

4 Authorize and submit a record in Swagger

Open the interactive docs. Click Authorize and paste your DISCHARGE_API_KEY from .env. Then POST the first sample record: open samples/records/appendectomy_record.json, copy it, and paste it into Try it out on POST /discharges.

OPEN IN YOUR BROWSER

```
PS> http://127.0.0.1:8000/docs
```

VALIDATE POST /discharges returns 201 with id 1, status submitted, and medications with integer dose_x1000. Without authorizing, the same call returns 401. Change one dose to 2.5 and resubmit: 422. GET /discharges lists the record with patient_initials and no name, MRN or phone.

SCREENSHOT SLOT Swagger: POST /discharges 201 response for PT-0001

5 Drive the clinician-in-the-loop journey

Run the whole review workflow by hand. Each call below proves one requirement. This is your 90-second demo.

#	Call	Body	Expect	Proves
1	POST /discharges/1/drafts	{"requested_by": "dr.mehta"}	201, version 1, pending_review	FR-04
2	POST /discharges/1/drafts	same	409 (a draft is already pending)	FR-04
3	POST /drafts/1/approve	{"reviewer_id": "dr.mehta"}	403 (submitter cannot review)	FR-10
4	POST /drafts/1/reject	{"reviewer_id": "dr.rao", "reason": "short"}	422 (reason too short)	FR-09

#	Call	Body	Expect	Proves
5	GET /discharges/1/patient-summary		409 (not approved yet)	FR-12
6	POST /drafts/1/approve	{"reviewer_id": "dr.rao"}	200, draft and record approved	FR-08
7	POST /drafts/1/approve	same	409 (decision is final)	FR-11
8	GET /discharges/1/patient-summary		200, name, med table, approval stamp; no MRN/DOB	FR-12

VALIDATE Every row behaves as listed. If any does not, that is a real bug: paste the call, the response, and the relevant route into Claude and fix it now. The core clinician-in-the-loop workflow works without any AI at all, which is exactly the point.

SPEC LINK Keep a running tick-list against SPEC §12. After this exercise FR-01, 02, 03, 04, 07, 08, 09, 10, 11, 12, 18, 19 and SEC-01 should be manually verified; FR-05 is partially done (the med table is real, the prose is a stub).

What success looks like

- Uvicorn serves the app and Swagger lists every endpoint in SPEC §11 that is in scope for this exercise.
- You ran submit, draft, reject, approve and release by hand and saw every guard (401, 403, 409, 422) fire.
- The drafter is a clearly-marked stub, deliberately deferred, not silently faked.
- The list endpoint and the patient summary never expose MRN, DOB or phone.

Common pitfalls (Windows and macOS)

- Port 8000 in use? Add `--port 8001` to the uvicorn command and open /docs on that port.
- Auto-reload only watches code. If you change `.env`, restart the server manually.
- A 422 on a body you think is valid usually means a strict type did its job, for example a quoted number in `dose_x1000`. Read the `detail` array; it names the field.
- If a test record already exists, POST /discharges returns 409 for the duplicate `patient_ref`. Change the ref or delete `dischargeflow.db` and restart.

Go deeper • AI developer track

Optional, for faster finishers. Each challenge extends the lab with a production-grade technique.

- Prove the race: fire two approvals for the same draft concurrently with `httpx.AsyncClient` and `asyncio.gather`. Expect exactly one 200 and one 409, and show which layer (unique constraint or status guard) actually won.
- Propose an `Idempotency-Key` header for POST /discharges as a SPEC change (changelog entry first), then implement and test it.

Stretch goal

Submit the second sample (`pneumonia_record.json`, submitted by dr.iyer) and ask Claude: "Which requirement in SPEC §6 is least well protected by the current code, and why?" Decide whether its answer is a code fix or a spec change.